Semantic Categorical Orchestra

The Conductor of a Research Operating System:
A Self-Contained Formalisation of an Orchestration Language

Kundai Farai Sachikonye
Technical University of Munich
Chair of Experimental Bioinformatics
kundai.sachikonye@wzw.tum.de

June 26, 2026

Abstract

We formalise an orchestration language for a research operating system. The system presents no applications and no windows: a blank surface on which a researcher writes a program in one language, and to which only the currently necessary result is returned. Specialised competence—testing, simulation, analysis, retrieval—is held not by the language but by a federation of *modules*, each a tool with its own domain-specific language that *terminates* its domain and answers within it. The orchestration language generates no general-purpose code; it compiles intent into the modules' own languages and routes it. This relieves the language of the burden of omniscience and raises a different problem, which is the subject of the paper. Every module, queried correctly, returns a *correct* answer; correctness is the module's guarantee. But a correct answer to the wrong question is worthless, and no module can tell whether its own task was the right one to perform, because that judgment is a property of the whole and a module is a bounded part. We prove, from a single derived fact—a strictly positive floor on the cost of telling any part from a non-completable whole—that necessity is unreadable from within any task (the No-Local-Necessity theorem), that it is therefore the irreducible function of a layer above the modules, and that this layer can certify necessity *without redoing any module's work*, by three measurements on the task-graph alone: a *contribution score* that vanishes exactly on purposeless tasks, a *holonomy* that detects when correct steps compose into a cycle that drifts from the stated intent, and an *order parameter* whose phase transition marks the ensemble's convergence onto a common goal. We define the language as a typed compiler from intent to module-language with a necessity gate at its centre, prove that the gate is sound (it never prunes a necessary task) and that orchestration converges to a region—an *action cell*—of admissible plans rather than to a single point, and we exhibit the conductor as the one component the federation cannot do without: a hall of flawless players is silent without it. Everything is proved of finite weighted graphs; minimum cuts are exact, and interpretation is confined to remarks.

Keywords: orchestration language; domain-specific language; research operating system; minimum cut; resolution floor; necessity; contribution score; holonomy; order parameter; bounded observer; action cell; separation of concerns.

Contents

Part I
Foundations: the floor, parts, and the whole

1 Introduction

1.1 A blank surface

Consider an operating system for research that shows nothing until asked. There is no desktop of applications, no array of windows, no file browser; there is a surface on which the user writes, and onto which the system returns the one piece of information the writing requires, after which the surface is clear again. Such a system replaces the application—a standing program waiting to be used—with a sentence: a request, expressed once, answered once. The question this paper addresses is what language that sentence is written in, and what that language must *be* in order for the system to work.

1.2 Players and a conductor

The competence of the system—the ability to test a piece of code, to simulate a reaction, to align two sequences, to retrieve a measurement—does not live in the language. It lives in a federation of *modules*: independent tools, each expert in one domain, each exposing its own small language by which it is addressed. A module is to its domain what a virtuoso is to an instrument. The orchestration language does not play the instruments; it does not generate the low-level code that a module would otherwise need handed to it. It compiles a request into the *module's own language* and dispatches it. The competence is the module's; the language only needs to know how to *ask*.

This division has an immediate benefit and a hidden cost. The benefit: the language need not be omniscient. It does not have to know how to write a correct simulation, only how to ask the simulation module for one; the simulation module's language already *terminates* that domain—it admits exactly the well-formed simulations and nothing else—so the orchestrator commits only the intent and lets the module's compiler enforce correctness. This is the classical separation of concerns (Dijkstra, 1972; Mernik et al., 2005) carried to its conclusion.

The cost is subtler, and it is the reason the orchestrator must *excel*, not merely route. Every module, asked a well-formed question, returns a *correct* answer; that is its contract. But a correct answer to a question that should never have been asked advances nothing. A hall of the finest players in the world, each rendering their part flawlessly, is noise—or silence—without someone to decide which piece is played, in what order, toward what whole. That someone is the conductor, and the conductor's skill is not a better rendition of any part. It is the judgment of *necessity*: whether each correctly performed task is the *right* task, a *needed* one, for the result the user actually asked for. We will prove that this judgment cannot be made by any player, and must be the defining function of the orchestrator.

1.3 The thesis

Three claims organise the paper, each proved below.

First (Sections 2 and 4), necessity is a global property and is invisible locally. We derive a strictly positive floor on the cost of individuating any part of a non-completable whole, and show that a bounded module, seeing only its own task against its own rest, cannot encode whether that task is necessary to the whole. Correctness is local and is the module's to guarantee; necessity is global and is not.

Second (Sections 5 to 8), the orchestrator can

nonetheless certify necessity without performing any module's work, by three measurements on the graph of tasks: a contribution score that is zero exactly on tasks whose removal changes nothing the ensemble can achieve; a holonomy that is nonzero exactly when correct tasks compose around a cycle that departs from the declared intent; and an order parameter whose value, and whose sharp transition, report how nearly the ensemble has converged on a single goal.

Third (Sections 9 to 11), the orchestration language is exactly a typed compiler from intent to module-language with a *necessity gate* at its centre. We prove the gate sound, prove that orchestration converges to a *region* of admissible plans rather than a single plan, and prove that the conductor is the unique component the federation cannot dispense with.

1.4 Method

The development is austere on purpose. Every object is a finite weighted graph or a construction on one. "Identity," "meaning," "necessity," "correctness" are each a specific graph quantity, defined before use, and minimum cuts are computable exactly by maximum flow (Ford and Fulkerson, 1956; Menger, 1927). Where a tempting claim could not be proved at this standard it has been removed rather than weakened, and every interpretive reading—of a module as a player, of the orchestrator as a conductor, of a task-cycle as a feedback loop—is confined to a remark on which no theorem depends. The paper is self-contained: a reader who grants only finite weighted graphs can check every claim without consulting any other work.

2 Contact graphs and the floor

We begin with the object on which everything rests and the one constant we will use throughout. Both are defined here from nothing.

Definition 2.1 (Finite weighted graph; cut)**.** A *finite weighted graph* is a triple $\Gamma = (V, E, w)$ with V a finite nonempty vertex set, $E \subseteq \binom{V}{2}$ undirected edges, and $w\colon E \to \mathbb{R}_{>0}$. For $\varnothing \neq S \subsetneq V$ the *cut* is $\partial(S) = \{\{u, v\} \in E : u \in S,\ v \notin S\}$, of weight $w(\partial(S)) = \sum_{e \in \partial(S)} w(e)$.

Definition 2.2 (Contact graph; medium; separation cost)**.** A *contact graph* has a distinguished vertex m, the *medium*, adjacent to every other vertex. The vertices $v \neq \mathsf{m}$ are *items*. The *separation cost* of an item v is the minimum weight of a cut placing v opposite the medium,

$$\sigma(v) = \min_{S \ni v,\ \mathsf{m} \notin S} w(\partial(S)),$$

the v–m minimum cut, attained on finite V and computable by max-flow (Diestel, 2017; Ford and Fulkerson, 1956; Menger, 1927). The minimising cut is the *resting cut* of v.

Remark 2.3 (The medium is "everything else")*.* The medium is not a place; it is the single vertex standing for all that an item is told apart from. Its universal adjacency encodes that every item is, at minimum, individuated against the whole rest. No theorem uses any reading of m beyond Theorem 2.2.

Definition 2.4 (Expanding family; non-completable whole)**.** An *expanding family* is a chain of contact graphs $\Gamma_0 \subseteq \Gamma_1 \subseteq \cdots$, each adding items and contacts while keeping the medium and all prior structure. The whole is *non-completable* if the family has no last stage: for every Γ_k there is $\Gamma_{k+1} \supsetneq \Gamma_k$. This is an order condition, not a cardinality assumption.

Theorem 2.5 (The floor)**.** *Let the whole be non-completable. Then there is a constant $\beta > 0$ with $w(e) \geq \beta$ for every contact and $\sigma(v) \geq \beta$ for every item: no item is told from the rest at zero cost, and there is no cut of weight* 0 *separating a nonempty item set from its nonempty complement.*

Proof. To identify an item v completely is to fix it by its cut against the entire rest of the whole. Suppose this needs zero residue: then at some finite stage Γ_k the whole has been brought to bear, so Γ_k contains the whole, contradicting non-completability (Theorem 2.4), which supplies a further stage with an item not yet weighed against v. Hence a positive residue remains at every finite stage; set β to its infimum. This residue is carried by the edges of v's resting cut, so each such edge has weight $\geq \beta$; every edge lies in some singleton cut, so every weight is $\geq \beta$; and connectivity forces $\sigma(v) = w(\partial(S^*(v))) \geq \beta > 0$. □

Remark 2.6 (Why a floor, and why it is not assumed)*.* β is not posited; it is the trace a non-completable whole leaves on each of its parts. A completable whole would permit zero-cost cuts and point-identities; the impossibility of finishing the comparison is exactly what makes the floor positive. Every later result depends only on $\beta > 0$, never on its value.

3 Identity, correctness, and the action cell

We now fix what "the identity of a part" and "a correct result" mean, and prove that each is a *region*, never a point. This is the structural fact behind every later claim about necessity.

Definition 3.1 (Weighted isomorphism; invariant)**.** A *weighted isomorphism* is a vertex bijection preserving the medium, edges, and weights. A quantity is a *graph invariant* if it agrees on isomorphic graphs.

Theorem 3.2 (Identity is the invariant cut, a region)**.** *The separation cost and resting cut of an item are graph invariants, while its label is permuted by automorphisms and is not. The identity of an item—the invariant datum fixing it—is its resting cut, a set of contacts of weight $\geq \beta > 0$; in general the minimising side $S^*(v)$ is not the singleton $\{v\}$. Identity is therefore a region of contacts, never a point.*

Proof. An isomorphism carries each v–m cut to an equal-weight cut and back, so the minimum is preserved; labels are permuted by any nontrivial automorphism. By Theorem 2.5 the cut has weight $\geq \beta > 0$. For non-locality, join two dense item-clusters by a single β-weight contact: the cheapest cut against m may peel off a whole cluster, a side of size > 1, below the cost of isolating any one vertex; the minimiser is then not a singleton. □

We carry this from a single item to a *result*: the outcome of a piece of work, scored by how far it stands from doing what was asked.

Definition 3.3 (Semantic distance; the floor of a result)**.** Fix a goal g (a designated item, the intended terminus). The *semantic distance* of a state x to g is the alignment gap

$$S(x) = \frac{\sigma(x, g)}{\Omega} \in (0, 1], \qquad \Omega := w(E),$$

where $\sigma(x, g)$ is the x–g minimum cut. By Theorem 2.5, $S(x) \geq \beta/\Omega =: S_\flat > 0$ for $x \neq g$: no state is perfectly on goal; $S_\flat$ is the irreducible floor of S.

Theorem 3.4 (Correctness is an action cell, not a point)**.** *The set $\mathsf{C} := \{x : S(x) = S_\flat\}$ of states at the floor—the* action cell *of the goal—is in general not a single state: distinct $x \neq x'$ may both attain $S_\flat$. A correct result is membership in C, a positive-volume region; it is never a unique prescribed output.*

Proof. $S(x) = S_\flat$ holds iff the x–g cut attains the floor. By Theorem 3.2 the floor-attaining cut need not be borne by a singleton, and two states whose cuts to g have equal floor weight are S-indistinguishable; construct them as in Theorem 3.2. Hence $|\mathsf{C}| > 1$ in general. □

Remark 3.5 ("Correct" is a region; this is not a defect). Theorem 3.4 says a module that returns *any* $x \in \mathsf{C}$ has returned a correct result; there is no single right output to match. This is why correctness can be a module's local guarantee and yet leave the harder question—whether the *goal* g was the one the user wanted—entirely open. We make that gap precise next.

4 No local necessity

This section proves the paper's pivot: a part cannot judge its own necessity. We first separate two notions kept apart throughout.

Definition 4.1 (Correctness versus necessity)**.** Let a *task* produce a result x aimed at a local goal g_t. The task is *correct* if $x \in \mathsf{C}(g_t)$: it does what it was asked, to within the floor. Given an *intent* g (the global goal) and an ensemble of tasks E, a task $t \in E$ is *necessary* if removing it strictly worsens the best semantic distance the ensemble can reach toward g:

$$\min_{x \in \mathrm{reach}(E \setminus \{t\})} S_g(x) \;>\; \min_{x \in \mathrm{reach}(E)} S_g(x),$$

where reach$(\cdot)$ is the set of states the ensemble can produce. Correctness is judged against g_t; necessity against g.

Definition 4.2 (Bounded module; its horizon)**.** A *module* is a bounded observer: a finite contact graph Γ_μ with a state capacity $\kappa_\mu = \log_2 |V_\mu|$ bits, individuating its task against *its own* rest m_μ, with no edge to the global goal g unless $g \in V_\mu$.

Theorem 4.3 (No local necessity)**.** *A module cannot, in general, determine the necessity of its own task. Precisely: if the global intent g lies outside the module's graph ($g \notin V_\mu$), then necessity—a predicate on the global ensemble's reach toward g—is not a function of any datum available within Γ_μ; and even when g is present, if the ensemble's global state complexity exceeds κ_μ bits, no correctness criterion encodable in the module decides it.*

Proof. (i) Absent goal. By Theorem 4.1 necessity quantifies over reach(E) and reach$(E \setminus \{t\})$, both

defined by cuts toward g. If $g \notin V_\mu$ there is no g-cut inside Γ_μ, so no internal quantity is a function of S_g; two global ensembles agreeing on Γ_μ but differing in whether t is necessary are indistinguishable to the module. Hence necessity is not module-internal.

(ii) Present goal, insufficient capacity. Suppose $g \in V_\mu$ but the global ensemble has N distinguishable necessity-configurations with $\log_2 N > \kappa_\mu$. A decision of necessity is a function from configurations to $\{0,1\}$ that must be *stored* (encoded) within the module's κ_μ bits to be applied; by the pigeonhole principle a κ_μ-bit state cannot label $N > 2^{\kappa_\mu}$ distinct configurations injectively, so some two configurations with opposite necessity collapse to one module-state and are decided alike—wrongly for one. No module-encodable criterion is correct on all of them. □

Corollary 4.4 (Necessity is the layer above)**.** *Necessity is decidable only by an observer whose graph contains the global goal g and the whole ensemble E, with capacity $\geq \log_2 N$. Since a module is by definition bounded to its own task, this observer is a distinct layer* above *the modules.*

Proof. Immediate from Theorem 4.3: the two failure modes are exactly absence of g and insufficiency of capacity; an observer holding g, E, and enough bits suffers neither, and is not any single module. □

Remark 4.5 (The orchestra, stated exactly). Theorem 4.3 is the formal content of the orchestra. Each player renders a correct part (membership in its own action cell, Theorem 3.4); no player can hear whether the *piece* is right, because the piece is a property of the whole ensemble's convergence on a goal that no part contains (Theorem 4.4). The conductor is not a better player; the conductor is the observer the theorem requires. No theorem depends on the reading.

Part II
The federation and the three measurements

5 Modules, languages, and the orchestrator

We now build the federation and the layer above it, and fix what each is allowed to do.

Definition 5.1 (Module; its language; termination of a domain)**.** A *module* μ is a pair $(\mathcal{L}_\mu, [\![\cdot]\!]_\mu)$ of a *domain-specific language* $\mathcal{L}_\mu$—a set of well-formed *instructions*—and an interpretation taking each instruction to a result. The module *terminates* its domain: $\mathcal{L}_\mu$ admits exactly the well-formed tasks of the domain and no others, and every admitted instruction is interpreted to a correct result ($x \in \mathsf{C}(g_t)$, Theorem 3.4). A module is a bounded observer (Theorem 4.2).

Definition 5.2 (Intent; the orchestrator)**.** An *intent* ι is a request written by the user in a single *orchestration language*, naming a global goal g. The *orchestrator* K (the conductor; *kwasa-kwasa*) is a map that takes an intent and a federation $\mathcal{M} = \{\mu_1, \ldots, \mu_k\}$ and returns a result surfaced to the user. K generates *no* domain instructions of its own beyond emitting, for each chosen subtask, a well-formed instruction in the relevant module's language $\mathcal{L}_\mu$. The competence of each domain is the module's; K's competence is elsewhere, and is the subject of Section 6 onward.

Principle 5.3 (Separation of competence). Two competences are kept disjoint. *Correctness* is each module's, enforced by $\mathcal{L}_\mu$'s termination of its domain (Theorem 5.1). *Necessity* is the orchestrator's, and by Theorem 4.3 can be nowhere else. The orchestrator never re-checks a module's correctness, and a module never judges its own necessity.

Remark 5.4 (Why the orchestrator emits a language, not code). A general code generator must hold the competence of every target domain—it must know how to write a correct simulation, a correct test, a correct alignment—which makes it an all-knowing part, contradicting boundedness and inviting error at every domain it touches. Emitting into $\mathcal{L}_\mu$ removes this: the orchestrator commits only the intent (which task, toward which goal), and the module's termination of its domain supplies correctness. The orchestrator is thereby freed to spend its whole capacity on the one thing no module can do—necessity. This is the classical compiler discipline of lowering to a typed target (Aho et al., 2006; Mernik et al., 2005) put to the service of Theorem 5.3; no theorem depends on the reading.

Definition 5.5 (Task graph)**.** For an intent ι the orchestrator forms a *task graph*: a directed graph $G_E = (E, A)$ whose vertices are tasks (each routed to a module) and whose arcs $u \to v$ record that v consumes u's result. Sources are the intent's givens; the designated sink carries the global goal

g. Each task t has a local goal g_t and, once run, a result $x_t \in \mathsf{C}(g_t)$.

The next three sections give the orchestrator's three instruments, each a measurement on G_E that needs no module re-execution.

6 Contribution: the necessity of a single task

Definition 6.1 (Ensemble floor; contribution score)**.** The *ensemble floor* is the best semantic distance the ensemble can reach toward g,

$$S_\flat(E) = \min_{x \in \mathrm{reach}(E)} S_g(x) \;\geq\; S_\flat.$$

The *contribution* of a task t is the increase in the ensemble floor caused by removing it,

$$\delta S(t, E) = S_\flat(E \setminus \{t\}) - S_\flat(E) \;\geq 0.$$

Theorem 6.2 (Contribution decides necessity)**.** *$\delta S(t, E) > 0$ iff t is necessary (Theorem 4.1); $\delta S(t, E) = 0$ iff t is* purposeless*—its removal leaves unchanged everything the ensemble can achieve toward g. Monotonicity holds: removing a task cannot lower the floor, so $\delta S \geq 0$ always.*

Proof. Removing a task can only shrink reach (fewer producible states), so the minimum over a subset cannot fall: $S_\flat(E \setminus \{t\}) \geq S_\flat(E)$, giving $\delta S \geq 0$. By Theorem 4.1 necessity is exactly $S_\flat(E \setminus \{t\}) > S_\flat(E)$, i.e. $\delta S > 0$; its negation $\delta S = 0$ is equality of the floors, i.e. purposelessness. □

Corollary 6.3 (A correct task may be purposeless)**.** *There exist tasks with $x_t \in \mathsf{C}(g_t)$ (perfectly correct) and $\delta S(t, E) = 0$ (purposeless): a flawless answer that the ensemble did not need. Correctness does not imply necessity.*

Proof. Let t produce a correct result on a local goal g_t disconnected from the sink g in G_E (its arc does not reach the sink). Then reach toward g is unchanged by its presence, so $\delta S = 0$ while $x_t \in \mathsf{C}(g_t)$. □

Remark 6.4 (The pruning the conductor performs). Theorems 6.2 and 6.3 give the conductor its first act: compute δS for each task and *prune* those with $\delta S = 0$. A purposeless task is correct and removable; pruning it changes nothing the ensemble can achieve and spares its cost. This is necessity exercised positively, by subtraction. Soundness—that pruning never removes a necessary task—is Theorem 9.3.

7 Holonomy: necessity around a cycle

Single-task contribution does not catch a subtler failure: tasks each correct and each necessary, composed around a feedback cycle that drifts from the goal. We measure the drift directly.

Definition 7.1 (Transport; specification; holonomy)**.** Each arc $u \to v$ of G_E carries a *transport* $T_{u \to v}$: the map by which v transforms u's result. For a directed cycle $c = (v_0 \to v_1 \to \cdots \to v_n = v_0)$ the *accumulated transport* is $T_c = T_{v_{n-1} \to v_0} \circ \cdots \circ T_{v_0 \to v_1}$. The intent supplies a *specification* T_c^{spec}, the transformation the cycle is declared to realise (the identity if the cycle should restore its starting state). The *holonomy* of c at a probe x is

$$\mathrm{hol}(c, x) = \| T_c(x) - T_c^{\mathrm{spec}}(x) \|.$$

Theorem 7.2 (Cycle inconsistency)**.** *If $\mathrm{hol}(c, x) > 0$ for some reachable probe x on some cycle c of G_E, then the ensemble is globally inconsistent with the intent: there is no assignment of results, each locally correct, whose composition around c matches the declared specification. Zero holonomy on every cycle is necessary for global consistency; on a DAG (no cycles) holonomy is vacuous and consistency is decided by the contribution scores alone.*

Proof. Composition around c returns to v_0; consistency with the intent requires the returned state equal the specified one, i.e. $T_c(x) = T_c^{\mathrm{spec}}(x)$ for all reachable x, i.e. $\mathrm{hol}(c, x) = 0$. A single x with $\mathrm{hol}(c, x) > 0$ exhibits a state the cycle transforms against specification; since each $T_{u \to v}$ is fixed by its (correct) module, no reassignment of *correct* results removes the mismatch—it lives in the composition, not in any node. Necessity of the condition is its definition; on a DAG there is no cycle, so the universally quantified condition is vacuously true and only Theorem 6.2 remains. □

Remark 7.3 (The route audit). Holonomy is an audit of the *route*, not the endpoints. Every node may be correct and every node necessary, yet the loop they form may accumulate a drift that no node reveals—a discrepancy visible only by transporting a probe all the way around and comparing with what was promised. Kirchhoff's loop law (Kirchhoff, 1847) and the holonomy of a connection (Kobayashi and Nomizu, 1963) are the same shape: a quantity that is consistent iff its accumulation around every closed loop vanishes. The

conductor's second act is to enumerate the cycles (by Johnson's algorithm (Johnson, 1975), after a strongly-connected decomposition (Tarjan, 1972)) and test each for holonomy. No theorem depends on the reading.

8 The order parameter: convergence on one goal

The third instrument measures, in a single scalar, how nearly the whole ensemble is pulling toward one goal, and detects the sharp moment at which it locks on.

Definition 8.1 (Phase; coupling; order parameter)**.** Assign each task t a *phase* $\theta_t \in [0, 2\pi)$: the direction, in goal-space, of its result's pull (its alignment angle toward g). Arcs couple phases with strength K proportional to shared contribution. The *ensemble order parameter* is

$$R\,e^{\mathrm{i}\psi} = \frac{1}{|E|}\sum_{t\in E} e^{\mathrm{i}\theta_t}, \qquad R \in [0,1],$$

the magnitude of the mean phase: $R = 0$ when pulls cancel (no shared goal), $R = 1$ when all pulls align (one goal) (Kuramoto, 1975, 1984; Strogatz, 2000).

Theorem 8.2 (Regimes and the locking transition)**.** *Under phase coupling on G_E there is a critical coupling K_c such that for $K < K_c$ the only stable state is incoherent ($R \approx 0$: the tasks do not share a goal), and for $K > K_c$ a coherent state with $R > 0$ emerges and grows. The order parameter stratifies the ensemble into regimes from* turbulent *(R small, no common action cell) through* coherent *(R near 1, converging on a common cell) to* phase-locked *($R = 1$); the onset at K_c is a genuine transition in the sense of Landau (1937).*

Proof sketch. The mean-field reduction of phase coupling (Kuramoto, 1984; Strogatz, 2000) gives a self-consistency equation $R = K\,R\,F(R)$ with F the coupling-weighted alignment kernel; $R = 0$ always solves it, and a second, positive solution bifurcates from it exactly when K exceeds $K_c = 1/F(0^+)$. Below K_c only $R = 0$ is stable; above, the coherent branch is stable and increasing in K. The free-energy expansion in the order parameter near K_c is that of a second-order transition (Landau, 1937), giving the sharp onset. Full coherence $R = 1$ is the fixed point at which all phases coincide. □

Remark 8.3 (A map, not a verdict). The conductor's third act returns not a pass/fail but a *regime map*: $(R, S_\flat(E), \mathrm{hol}, \delta S)$. By Theorem 3.4 "correct" is already a region, so the honest report of an ensemble's health is likewise a region-valued characterisation—how coherently the tasks converge—rather than a point verdict. A low R says the ensemble is doing many correct things that do not add up to one goal; that is precisely a failure of *necessity*, made quantitative. No theorem depends on the reading.

Part III
The language, the cell of plans, and the conductor

9 kwasa-kwasa as a typed compiler with a necessity gate

We now assemble the three instruments into the orchestration language and prove its central property: it judges necessity soundly while never touching correctness.

Definition 9.1 (The orchestration pipeline)**.** K maps an intent ι (global goal g) to a surfaced result by:

(1) *Decompose*: form a task graph G_E (Theorem 5.5), routing each task to a module whose domain it falls in.

(2) *Lower*: emit, for each task, a well-formed instruction in its module's language $\mathcal{L}_\mu$ (Theorem 5.4); correctness is the module's (Theorem 5.1).

(3) *Gate*: judge necessity on G_E—prune tasks with $\delta S = 0$ (Theorem 6.2), reject ensembles with $\mathrm{hol} > 0$ on any cycle (Theorem 7.2), and report the regime R (Theorem 8.2).

(4) *Surface*: return the result of the gated, coherent ensemble; clear the surface.

Definition 9.2 (Well-typed orchestration)**.** A task is *well-typed* if its instruction is well-formed in its module's language and its arcs connect output types to input types along G_E. K *accepts* an ensemble iff it is well-typed, has zero holonomy on every cycle, and contains no purposeless task after pruning.

Theorem 9.3 (Soundness of the necessity gate)**.** *The gate is* sound*: it never prunes a necessary task and never rejects a globally consistent ensemble. Formally, if t is necessary (Theorem 4.1) then the gate retains it; if E is globally consistent with the intent (zero holonomy on every cycle, every task necessary) then the gate accepts it.*

Proof. The gate prunes t only when $\delta S(t, E) = 0$, which by Theorem 6.2 is exactly purposelessness, the negation of necessity; so a necessary task ($\delta S > 0$) is never pruned. The gate rejects only on $\mathrm{hol} > 0$ for some cycle, which by Theorem 7.2 is exactly global inconsistency; so a consistent ensemble (all holonomies zero) is never rejected. Both gate actions are characterisations, not heuristics, so each is sound by the theorem it invokes. □

Theorem 9.4 (The gate touches only necessity, never correctness)**.** *The gate's three measurements—δS,* hol*, R—are functions of the task graph G_E and the results x_t alone, and never re-evaluate whether $x_t \in \mathsf{C}(g_t)$. Correctness is read from the module's contract and treated as given; the gate decides only necessity and consistency.*

Proof. δS depends on reach and $S_\flat(E)$ (Theorem 6.1); hol on the transports and the specification (Theorem 7.1); R on the phases (Theorem 8.1). Each is computed from G_E and the delivered results; none recomputes a module's local goal g_t or re-checks $x_t \in \mathsf{C}(g_t)$, which is supplied by Theorem 5.1. Hence the gate is orthogonal to correctness, realising Theorem 5.3. □

Remark 9.5 (Why the language can be thin)*.* Theorem 9.4 is the reason the orchestration language is small. It carries none of the domains' competence—no module's grammar, no domain correctness—only the three graph measurements and the routing. A user learns to *express intent and route it*, not to write simulations or tests; the surface of the language is the entire interface to the system, and it is thin precisely because competence lives below it. No theorem depends on the reading.

10 Orchestration converges to a region of plans

We show that the gate does not seek a unique plan—there is none to seek—but relaxes the ensemble into an *action cell of plans*, and that this relaxation halts or declines.

Definition 10.1 (Plan; demand; relaxation)**.** A *plan* is an accepted ensemble (Theorem 9.2). Its *demand* is $D(E) = S_\flat(E) - S_\flat$, the ensemble's distance above the irreducible floor. The *relaxation* repeatedly: prunes a purposeless task, repairs a holonomy violation by re-routing the offending cycle, or adds a task that strictly lowers $S_\flat(E)$—each step a committing change to G_E.

Theorem 10.2 (Monotone relaxation; quiescence or decline)**.** *Along the relaxation the demand $D(E)$ is non-increasing and bounded below by* 0*. The relaxation either reaches* quiescence*—a plan whose demand cannot be lowered, the ensemble resting in the action cell of the intent (Theorem 3.4)—in finitely many steps, or it does not halt, in which case no plan attains the intent and the honest output is to* decline *rather than surface a non-converged ensemble.*

Proof. Each step either removes a task that does not raise $S_\flat$ (pruning, D unchanged or lower), repairs a cycle (restoring consistency without raising $S_\flat$), or adds a task that strictly lowers $S_\flat$; none raises D, so D is non-increasing, and $D \geq 0$ by Theorem 3.3. The space of ensembles over finitely many available tasks and routings is finite, so a non-increasing demand either attains its infimum at a finite step—quiescence, a fixed point in the sense of Banach (1922); Knaster (1928); Tarski (1955)—or the relaxation cycles without lowering it, never reaching the floor. In the latter case $D(E) > 0$ for every reachable plan, so no plan lands in C; surfacing any of them would report attainment that does not hold, and declining is the only output consistent with the floor. □

Theorem 10.3 (The plan is a region, certified blind)**.** *At quiescence the set of admissible plans is in general a positive-volume region, not a single plan: distinct accepted ensembles may share the minimal demand. The orchestrator certifies a plan by the* absence of any lowering move*—a property of the demand—without ever reading the meaning of any module's result.*

Proof. By Theorem 3.4 the floor of S is attained on a region; distinct ensembles reaching that region have equal minimal demand and are gate-indistinguishable, so the admissible set has > 1 element in general. Acceptance tests only $\delta S, \mathrm{hol}, R$ (Theorem 9.4), all functions of the graph and the delivered results, never of an interpreted "meaning" of a result; hence the certification is blind. □

Remark 10.4 (Why blind certification is the right standard). The orchestrator cannot read what a module's result *means*—that would require occupying the module's entire domain, i.e. being the module, collapsing Theorem 5.3. It does not need to. Necessity and consistency are graph-level facts about contribution, holonomy, and coherence, decidable without interpretation. The conductor judges whether the parts *add up*, not what each part privately is. No theorem depends on the reading.

11 The conductor is indispensable

We close with the structural statement that motivates the whole construction: the federation cannot do without the orchestrator, and the orchestrator's indispensable function is exactly necessity.

Theorem 11.1 (Indispensability of the conductor)**.** *Let a federation $\mathcal{M}$ of modules be given, each individually flawless (every delivered result correct, Theorem 5.1). Without the orchestration layer:* (i) *purposeless tasks are undetectable, since no module can compute δS over the global ensemble (Theorem 4.3);* (ii) *cyclic inconsistency is undetectable, since holonomy lives in the composition and in no node (Theorem 7.2);* (iii) *convergence on a single goal is unmeasured, since R is a property of the ensemble, not of any task (Theorem 8.1). Hence a federation of correct modules, however excellent, cannot certify that its work is necessary, consistent, or convergent: it can be a hall of flawless players producing no coherent result.*

Proof. Each of (i)–(iii) is a global quantity—a function of G_E and g—and by Theorems 4.3 and 4.4 no bounded module computes a global necessity predicate. A federation is exactly a set of bounded modules; absent a layer holding g and E, none of the three measurements is available, so none of the three failures is detectable. Correctness of every part is consistent with all three failures by Theorem 6.3 and Theorem 7.2. Thus the federation alone cannot certify its own coherence. □

Theorem 11.2 (Master equivalence)**.** *For a federation orchestrated toward a non-completable whole, the following are equivalent:*

(i) *there is a floor $\beta > 0$ on individuating any part of the whole (Theorem 2.5);*

(ii) *correctness is a region (an action cell), not a point (Theorem 3.4);*

(iii) *necessity is not decidable within any bounded module (Theorem 4.3);*

(iv) *judging necessity, consistency, and convergence is the irreducible function of a layer above the modules—the conductor (Theorems 4.4 and 11.1);*

(v) *the orchestration language is a thin, typed compiler that lowers intent into module languages and gates only on the three graph measurements, certifying plans blind and to within the floor (Theorems 9.3, 9.4 and 10.3).*

Each is the same fact: telling a part from a non-completable whole costs a positive, irreducible residue; that residue makes correctness a region and necessity global; a global judgment cannot be made by a part; so it is made by the conductor, whose language need carry nothing but the judgment.

Proof. (i) ⇒ (ii) is Theorem 3.4; (ii) ⇒ (iii) holds because a region-valued correctness leaves the global goal underdetermined locally, the content of Theorem 4.3; (iii) ⇒ (iv) is Theorem 4.4 with Theorem 11.1; (iv) ⇒ (v) is the construction of Sections 9 and 10; and (v) ⇒ (i) because the gate's measurements are defined through $S_\flat = \beta/\Omega > 0$, whose positivity is the floor. The cycle closes. □

11.1 Computational note

Every quantity above is computable on finite weighted graphs without interpretation: minimum cuts (hence σ, S, $S_\flat$, δS) by maximum flow (Ford and Fulkerson, 1956; Menger, 1927); cycles for holonomy by strongly-connected decomposition (Tarjan, 1972) and elementary-circuit enumeration (Johnson, 1975); the order parameter by the mean-phase sum (Theorem 8.1); and the regime by the mean-field threshold (Theorem 8.2). The contribution score requires, in the worst case, one ensemble-floor evaluation per task (ablation), which the static measurements hol and R do not; a regime map can thus be produced before paying the ablation cost.

11.2 Scope

Every result is a theorem about finite weighted graphs and the constructions on them, provable from Theorem 2.1 and the derived floor Theorem 2.5. The development establishes *that* correctness is local and region-valued, *that* necessity

is global and not locally decidable, *that* an above-the-modules conductor is required and suffices, and *that* the orchestration language is exactly a thin gated compiler. It does not prescribe the internal design of any module or the syntax of any module language; those are the federation's, by Theorem 5.3. One hypothesis beyond the graphs is used: that the whole is non-completable (Theorem 2.4), an order condition, not a cardinality posit, and exactly what forces the floor; absent it, the floor may vanish and the separation of competence is not compelled. Interpretive readings—of modules as players, of the orchestrator as a conductor, of a blank surface as the user interface, of holonomy as a feedback drift—are confined to remarks on which no theorem depends, and material that could not be proved at this standard has been omitted.

11.3 Conclusion

A research operating system that shows nothing until asked needs one language to be asked in, and that language need not be wise. Wisdom—the competence of each domain—belongs to a federation of modules, each terminating its domain in its own tongue and answering correctly within it. What such a federation cannot supply, from a theorem rather than an oversight, is the judgment of whether its correct answers were the *needed* ones: necessity is a property of the whole, and a bounded part cannot read the whole. The orchestration language is therefore not a better player but the conductor: it lowers a researcher's intent into the modules' own languages, and then, on the graph of tasks alone, prunes what contributes nothing, rejects what drifts around a cycle, and reports how nearly the work converges on one goal—certifying the plan blind, to within the floor, never to a point. Its excellence is concentrated where no module can help: in telling the right work from the merely correct. A hall of the finest players is silent without it; with it, the silence of the surface between requests is the sign that exactly the necessary work, and no more, was done.

References

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.

Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.

Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017.

Edsger W. Dijkstra. The humble programmer. *Communications of the ACM*, 15(10):859–866, 1972.

L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.

Donald B. Johnson. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, 4(1):77–84, 1975.

Gustav Kirchhoff. Über die auflösung der gleichungen, auf welche man bei der untersuchung der linearen vertheilung galvanischer ströme geführt wird. *Annalen der Physik*, 148(12):497–508, 1847.

Bronisław Knaster. Un théorème sur les fonctions d'ensembles. *Annales de la Société Polonaise de Mathématique*, 6:133–134, 1928.

Shoshichi Kobayashi and Katsumi Nomizu. *Foundations of Differential Geometry, Volume I*. Interscience Publishers, 1963.

Yoshiki Kuramoto. Self-entrainment of a population of coupled non-linear oscillators. In *International Symposium on Mathematical Problems in Theoretical Physics*, volume 39 of *Lecture Notes in Physics*, pages 420–422. Springer, 1975.

Yoshiki Kuramoto. *Chemical Oscillations, Waves, and Turbulence*. Springer, 1984.

Lev D. Landau. On the theory of phase transitions. *Zhurnal Eksperimental'noi i Teoreticheskoi Fiziki*, 7:19–32, 1937.

Karl Menger. Zur allgemeinen kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927.

Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.

Steven H. Strogatz. From kuramoto to crawford: Exploring the onset of synchronization in populations of coupled oscillators. *Physica D*, 143 (1–4):1–20, 2000.

Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2): 146–160, 1972.

Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.